

微调影响转换吗？

RKLLM 转换原理 × 微调对接 完全指南

Qwen3 / DeepSeek-R1-Distill / Qwen2-VL 微调产物上板部署

配套脚本：`scripts/export.py` `export_qwen3.py` `finetune_classifier.py`

官方参考：github.com/airockchip/rknn-llm

2026-08-11

目录（按你的疑惑组织）

- 1 直接回答：微调影响转换吗？（一张总表）
- 2 转换器眼中，模型只有三样东西——为什么微调“本质无影响”
- 3 逐环节走查：微调在 加载/构建/导出 各环节起什么作用
- 4 有影响的 4 个地方（决定成败，必须处理）
 - 4.1 LoRA 怎么交付给转换器（合并 vs model_lora）
 - 4.2 词表/特殊 token 变了怎么办
 - 4.3 模型架构必须保持不变
 - 4.4 校准数据要与微调后的行为分布匹配
- 5 完全没有影响的 3 件事（可以放心）
- 6 验收方法：怎么证明微调效果真的进了 .rkllm
- 7 常见误区与典型错误
- 8 你的脚本逐行对照：微调版转换该怎么写
- 9 参考资源

1. 直接回答：微调影响转换吗？

一句话：微调【不影响转换机制本身】，但【影响 4 个必须处理的对接点】。

转换器只做机械的翻译工作——把已有权重重新打包成 NPU 格式。它不学习、不训练，不关心权重是预训练的还是微调出来的。只要微调后的模型还能被标准 transformers 加载，转换就能照常进行，微调学到的能力会完整保留在 .rkllm 里。

但“能转”不等于“转对”——有 4 个环节微调如果不处理，转换结果会等于没微调、或质量变差。总表如下：

转换环节	微调有影响吗	原因 / 后果
加载模型 load_huggingface	有（看 LoRA 怎么交付）	只给主模型=微调丢失；要给合并模型或 model_lora 参数
模型架构解析	必须保持不动	改架构（如加层、改注意力）可能超出 RKLLM 支持的算子集，直接转换失败
词表 / tokenizer	有（加了新 token 时）	vocab_size 不一致 → 报错或乱码
量化 build（校准）	有（间接）	校准数据要和微调后的真实使用场景分布一致，否则激活量化不准
量化 build（算法/精度）	无	W8A8/W4A16/GRQ 是纯数值压缩，与权重来源无关
算子融合 optimization_level	无	纯性能优化，与内容无关
导出 export_rkllm	无	打包写出，机械步骤
板端部署 / 推理	无	流程完全一样，只是 .rkllm 内容不同

记忆口诀：微调影响【给什么】和【校准数据】，不影响【怎么转】。

2. 转换器眼中，模型只有三样东西

RKLLM-Toolkit 读取 HF 模型时，只关心三样东西，微调与这三样东西的关系决定了所有影响：

转换器要的	来自哪里	微调有没有动它
① 架构描述	config.json (hidden_size、层数、注意力头数、RoPE 参数等)	LoRA 微调完全不动它 → 无影响
② 权重数值	safetensors / pytorch_model.bin	微调唯一改变的东西 → 转换打包的就是它
③ 词表	tokenizer.json / tokenizer.model + vocab_size	一般不动；只有加了新 token 才动 → 需检查

所以原理很清楚：微调改的只是【②权重数值】——而转换的全部意义就是打包权重数值。微调权重怎么进入转换，是唯一需要你操作的对接点，其余全是机械流程。

2.1 转换四步走（每一步跟微调的关系）

步骤	做了什么	微调的影响
① 结构解析	按 config.json 重建计算图，把算子映射到 NPU 算子集	LoRA 不动架构 → 无影响
② 权重重排	把 HF 张量重排成 NPU 布局（reshape/permute/分块）	纯机械变换 → 无影响
③ 量化	FP16→INT8/INT4，附 scale 参数；激活量化需跑校准	权重来源无关；激活校准与场景有关
④ 算子融合	合并相邻算子，减少 NPU 调用	与权重内容无关 → 无影响

3. 逐环节走查：微调在三个阶段起什么作用

3.1 加载阶段：微调成果在这里进（或丢）

load_huggingface 只认两种输入，微调成果必须通过其中之一进入：

```
# 输入 A: 合并后的完整 HF 模型（推荐，最稳）llm.load_huggingface(model='./model/Qwen2-VL-2B-OCR-merged',
model_lora=None, device='cpu', dtype='float16') # 输入 B: 主模型 + LoRA adapter，官方自动合并
llm.load_huggingface(model='./model/Qwen2-VL-2B-Instruct', model_lora='./lora_output', device='cpu',
dtype='float16')
```

如果只给主模型、model_lora 又不传：转换全程成功、无报错，但 .rkllm 里是原版模型——微调等于没做过。这是唯一“悄悄失败”的环节，验收时必须验证（见第 6 章）。

3.2 构建阶段：量化是纯数值操作，但校准数据受微调影响

build 做的量化（W8A8/W4A16）是对权重和激活做数值压缩，不关心数值是怎么来的。但 w8a8 的激活量化需要【校准】：把样本喂给模型，统计激活值范围。校准样本如果用微调前的旧场景（比如通用问答），而模型微调后变成了 OCR 专用，两者激活分布差异大，量化出来的精度会打折扣。

3.3 导出阶段：无影响

export_rkllm 只是把构建结果写成一个自包含的 .rkllm 文件。任何模型在这步都没有区别。

4. 有影响的 4 个地方（决定成败）

4.1 LoRA 怎么交付给转换器

方式 A：微调后合并再保存（推荐）

```
from peft import PeftModel model = PeftModel.from_pretrained(base_model, './lora_output') merged =
model.merge_and_unload() merged.save_pretrained('./model_merged') tokenizer.save_pretrained('./model_merged')
# 必须一起存 # 转换时：load_huggingface(model='./model_merged')
```

方式 B：不合并，转换时挂 adapter

```
llm.load_huggingface(model=base_model_path, model_lora='./lora_output', # 官方读 adapter 合并 device='cpu',
dtype='float16')
```

判断自己属于哪种：看微调脚本保存的是什么。如果保存的是 adapter_config.json + adapter_model.safetensors（PEFT 默认），就按方式 A 合并或方式 B 传参，二选一。

4.2 词表 / 特殊 token 变了怎么办

如果微调时调用过 add_special_tokens 或改过 tokenizer：

- 重新保存 tokenizer 到模型目录，保证 config.json 的 vocab_size == tokenizer.vocab_size

- 新增 token 的 embedding 是随机初始化的：转换会把它原样打包，若该 token 用得少、训练不足，量化后可能放大噪声 → 建议微调时充分覆盖，或转换前用 merge_and_unload 后重新统计
- RKLLM 按 config.json 读词表，对不上会报错或生成乱码

4.3 模型架构必须保持不变（唯一可能导致“转不了”的情况）

LoRA 微调 (peft 默认) 只插入低秩旁路，不改任何层结构，架构完全不变 → 安全。

但如果你的微调改了结构（重新实现 modeling 文件、加层、改注意力实现），转换器按 config.json 重建图时可能映射不出对应算子 → Build failed。判据：微调后的模型用 transformers 能正常 load 和推理，架构就是自洽的；标准 Qwen/DeepSeek 系列 + peft LoRA 不存在这个问题。

4.4 校准数据要与微调后的行为分布匹配

- 校准样本条数：32~128 条为宜（你现在的 data_quant.json 19 条偏少）
- 内容：与板上实际要跑的输入同分布。OCR 微调模型就用带图片描述的 OCR 文本样本，而不是通用百科
- input 要带对话模板 (Human: ... \nAssistant: ... 或对应 chat template)，和推理时一致

5. 完全没有影响的 3 件事（放心）

放心项	为什么
量化精度方案 (w8a8/w4a16/GRQ 选择)	纯数值压缩，对任何权重的压缩方式一样；微调权重和预训练权重在压缩面前地位相同
优化等级、NPU 核数、max_context 等性能参数	只影响速度和内存，与模型内容无关
部署流程 (拷贝 .rkllm → rkllm_init → rkllm_run)	与模型来源完全无关；微调版和原版部署方式一字不差

换句话说：如果你已经能把原版模型跑上板，微调版只多一步——保证 4.1 的 LoRA 交付正确。其余流程、脚本、配置可以原样复用。

6. 验收方法：怎么证明微调效果真的进了 .rkllm

因为“悄悄失败”是可能的（见 3.1），转换前后必须做验证，按顺序：

步骤	做法	通过标准
① PC 上验证微调产物	transformers 加载合并后的模型，跑一条真实输入	输出确实是微调后的效果（不是原版行为）
② 转换日志确认	转换时观察终端日志	出现加载 lora / 合并 adapter 的相关输出（方式 B 时）
③ 板端对比测试	把同一句输入分别喂给 板端 .rkllm 和 PC 微调模型	语义一致（允许量化导致的细微差异），而不是回到原版行为
④ 体积抽查	对比转换前后 .rkllm 文件大小是否合理	异常偏小可能说明权重没加载全

验收输入要选【微调特有】的样本：比如 OCR 微调模型，就用你训练集里那种带噪图片的文本，而不是“你好”这类通用问候——只有微调特有样本才能区分“带了微调”和“没带微调”。

7. 常见误区与典型错误

误区 / 错误	真相 / 解决
微调过就不能用官方转换了	完全不影响。官方工作流就是“外部微调 → 官方转换”，微调产物是标准 HF 格式，转换器直接吃
只保存了 LoRA adapter 就够	不够。转换器需要完整权重：要么合并保存，要么转换时传 model_lora，必须二选一
转换 = 重新训练，会把微调效果冲掉	转换不学习不训练，权重原样打包，微调效果完整保留
量化会重新拟合权重	不会。量化只是把数值压缩存储（附 scale），不做任何学习
校准数据越多越好	多不如对。要贴合目标场景，且 input 必须带对话模板；40+ 条、分布对，胜于 500 条无关数据
转换失败 = 模型坏了	先查环境：内存、平台名、dataset 路径、dtype。绝大多数 build 失败是环境问题
转了 w8a8 效果差 = 微调有问题	先怀疑校准数据；再试 w8a8_gx / hybrid_rate=0.2 / optimization_level=0 对比定位

8. 你的脚本逐行对照：微调版转换该怎么写

8.1 现状：export.py / export_qwen3.py（原版模型转换）

```
# 你的 export.py 关键行：llm.load_huggingface(model=modelpath, model_lora=None, device='cpu',
dtype='float16', custom_config=None, load_weight=True) # ^^^ 目前 None = 用原生权重
llm.build(do_quantization=True, optimization_level=1, quantized_dtype='W8A8', quantized_algorithm='normal',
target_platform='RK3588', num_npu_core=3, # ← 板子是 RK3576 要改 extra_qparams=None,
dataset='./data_quant.json', hybrid_rate=0, max_context=4096)
```

这段脚本本身没有微调概念——它把目录里的权重原样打包。要让微调成果进 .rkllm，只改 load 那一行（二选一）：

8.2 微调版：只需改 model 路径或 model_lora

```
# 写法一：先合并再转（配 finetune_classifier.py 的产物）from peft import PeftModel base =
'/home/gary/RK3576/rknn/05_llm/model/Qwen2-VL-2B-Instruct' model = PeftModel.from_pretrained(base,
'./model/Qwen2-VL-2B-OCR') merged = model.merge_and_unload()
merged.save_pretrained('./model/Qwen2-VL-2B-OCR-merged')
tokenizer.save_pretrained('./model/Qwen2-VL-2B-OCR-merged')
llm.load_huggingface(model='./model/Qwen2-VL-2B-OCR-merged', model_lora=None, device='cpu', dtype='float16') #
其余 build / export 行完全不变 # 写法二：不合并，转换时挂 adapter llm.load_huggingface(model=base,
model_lora='./model/Qwen2-VL-2B-OCR', device='cpu', dtype='float16')
```

两个注意点：① Qwen2-VL 是多模态模型，官方支持列表里有 Qwen2-VL/Qwen3-VL 架构，但 build 时需确认 vision 部分处理方式（多模态转换另见官方 multimodal 示例）；② RK3576 板子记得 target_platform='rk3576'、num_npu_core=2。

9. 参考资源

资源	位置
官方仓库（源码/文档/示例）	github.com/airockchip/rknn-llm（国内镜像 gitee.com/airockchip/rknn-llm）

资源	位置
官方 SDK 完整包（含中文手册 PDF）	RKLLM_SDK: console.zbox.filez.com/l/RJJDmB 取件码 rkllm
官方用户手册	仓库 doc/Rockchip_RKLLM_SDK_CN_1.3.0.pdf
已转换模型库	rkllm_model_zoo: console.box.lenovo.com/l/l0tXb8 取件码 rkllm
多模态部署示例	官方仓库 examples/multimodal_model_demo（Qwen2-VL 类模型参考）

最终结论：微调影响【给什么】（LoRA
交付方式）和【校准数据】（分布匹配），不影响【怎么转】（机制、量化、优化、部署全流程）。把第 4
章四个点处理好，微调成果就能完整、高质量地进入 .rkllm。